How "lights and moves" fits the build (2026-10-13)

What this is. Karl's idea (Appendix A/B of the prompt, ideas 1-8) mapped onto the code as it stands on commit `7d1485f`. Read with `CURRENT-STATE.md` / `.html`, which has the screenshots cited here (`add-11` = `shots/add-11-…webp`). Nothing was built or changed. **Observed** = seen in a shot or in a file and line; **inferred** = my reading, with the reason.

Verdict scale: already built · small change · large change · conflicts (with the loop spec or another idea) · unclear.

Summary table

#	Idea	Verdict	Main files
1	Lights beside moves	**Mostly built** (layout); the "click a red light to reveal its move" step and real rules for most lights are missing	`src/view/lightScreen.js:253-300`
2	Red / yellow / green progress lights	**Large change, and conflicts** with loop §5 and the decision log (red today means "that move made it worse")	`addBothMoves.js:80-88`, `divideMoves.js:227`, `lightScreen.js:287`
3	Icons at every level; inactive ones greyed and dead	**Mostly built**; "finished move greys out" is new (small); greyed moves today answer a tap with a message (loop §5, Open 9)	`lightScreen.js:263-280`
4	Preview of the move on the equation (hover; first tap on touch)	**Large change** (nothing exists); conflicts with loop §5's touch rule and, for Divide, with "always show all five fractions"	`solveScreen.js` (row builder), `lightScreen.js`
5	Fill the blanks in the equation; Go merged into Next	**Large change**; partly conflicts with "the answer is never shown before it is typed" (loop §2, §1C) when the system combines automatically	`addBothMoves.js:393`, `solveLoop.js` (`commit`, `fold`, `foldNext`)
6	No Guess x / Use a move choice; `x = [ ]` always available	**Conflicts** with loop §1A (its core), §1B, conformance item 1 and both card specs; the code change itself is medium	`solveLoop.js` (`path`, `answerLive`, `blockMove`), `addBothMoves.js` (`path`)
7	Scaffold space only when needed	**Piloted** (`?layout=tucked` on Divide it; its two bugs fixed 2026-10-15); Add already reserves none but blanks the stack (against loop §6)	`src/view/tuckedLayout.js`, `divideScreen.js:105-160`
8	A colour (and shape) per move	**Small to medium change in code, conflicts** with loop §3D ("the operation is blue everywhere") and the colour budget	`lightScreen.js:32`, `style.css:2846`, `2970` + the pulsing-target blue

The two Karl ranked first (4 and 5) are the two largest changes, and they depend on a third decision the design chat has not made: whether the moves are live before any choice (idea 6).

See §6 and the questions.

Part 2, report 1: which ideas already exist

- **Idea 1, lights beside moves: built as a layout.** The Why? boxes sit in a grid beside the moves they go with, each group centred on its moves' rows (`WHYS`, `lightScreen.js:253-259`; `add-01`). The pairings Karl remembers are in the code: Fewer terms ↔ Add and Cross the border; No fractions, Smaller numbers, One box ↔ Take a fraction and Multiply; No opposite box ↔ Flip everything; x on the left ↔ Trade places; Each side simplified ↔ Distribute and Combine; Solved for x on its own row. **What is missing for "the light has its move beside it":**
- **Lights are not clickable.** Message 1 says clicking a red light reveals its move's icon beside it, and clicking the icon opens the move. Today the icon is always there and the light does nothing. (Appendix B #1 says the icon is always beside the light, which is what's built. These two readings differ; see Q6.)
- **The pairing is one-to-many in both directions**, so "the move beside a light" is not always one move: Fewer terms has two moves beside it, and the fraction/multiply pair shares three lights. A light that "points at its move" has to choose one.
- **Most lights have no rule** (CURRENT-STATE §1.3): on Add, six lights simply copy Fewer terms' green (`lightScreen.js:287`; `add-11`); on Divide they are forced off.
- **The tucked pilot moved the other way:** lights in their own column, moves hidden in a drawer (`divt-01`, `divt-02`). If idea 1 is the direction, the tucked layout's split of lights and moves should not become the default as is.
- **Idea 3, icons at every level: built.** All eight icons show on every level of both cards, greyed except the card's move (`add-01`, `div-01`, L1 included: `add-51`, `div-39`). Hovering a greyed one does nothing (`add-04`). Two differences: a tap on a greyed move today prints a line message ("That move unlocks on a later card."; `add-05`), and the card's own move stays live after its job is done (nothing greys it out when its light is green).
- **Idea 7, scaffold only when needed: piloted** as `?layout=tucked` (Divide only): the counters tray slides up for I'm stuck / Teach me and the moves live in a drawer (`divt-24` to `divt-27`). Two bugs found: the tray clips the diagram at 1366×768, and after a move the keyboard can no longer type into `x =` (focus lands on the Moves tab; `src/main.js:541` ignores keys while a non-pad button has focus). **Both fixed on 2026-10-15.** The classic Divide screen keeps an empty 350×215 box for counters (`div-01`); Add keeps no box but its scaffold **replaces the work stack** (`add-26`), which loop §6 calls a bug.
- **Ideas 2, 4, 5, 6, 8: not built in any form.** The nearest things: Add's grey "−" Fewer terms after a partly cancelling move (`add-20`) is the only in-between light; Divide's Next-to-fold after a partial move (`div-36`, `div-37`) is the nearest thing to "Next commits"; Add L1's numeric level already has the student apply the amount to each side in turn (`add-52` to `add-54`).

Part 2, report 2: a partly-done (yellow) state, defined by a count

What it takes (inferred from the code): - **Lights must become a function of the equation, not of the move.** Today they are set by the move that was just made: Add compares before and after (`judge`, `addBothMoves.js:84`), Divide hard-codes `smaller: 'green'` on any successful fraction (`divideMoves.js:227`). A count needs one shared function per light, `count(board) → number left`, run on every state. Loop §8 already asks for this ("lights are computed on every state"). - **The board must be able to describe what the lights count.** The Add board knows terms and x; the Divide board knows a coefficient and fractions of a part (`solveTerms.js`). Neither has parentheses or opposite boxes yet, so several counts below can only be written when those cards exist. - **A fourth visual state** (yellow plus a shape, so colour is never the only signal: e.g. ✓ / ◖ / ×) in `checkbox()` (`lightScreen.js:246`) and the CSS (`style.css:2680-2684`). - **Small change in drawing, medium in rules, plus a spec change** (below).

Proposed counts, per light:

Light	Proposed count (0 = green)	Yellow when	From the equation state?
Fewer terms	number of terms on x's side other than x, plus x-terms on the other side	the count fell but is not 0	**Yes.** Add's `xTerms` already does a version of it (`addBothMoves.js:80`). **On one-step cards the count is only 0 or 1, so yellow never shows.**
One box	how many times the coefficient can still be shared down (prime factors of `|a|`: 6 → 2, 3 → 1, 1 → 0)	e.g. `6x = 12` → `3x = 6`	**Yes** (Divide's board has the coefficient). Divide L5 (`4x`, `6x`) is the only place a one-step card would show yellow.
Smaller numbers	**unclear.** "As small as they can be" has no natural count for `ax = b`; the honest version is "divide everything by the largest common factor", which is the same event as One box on these cards	—	Partly. Needs a definition (open; not asked yet).
No fractions	number of fraction terms left	some cleared	**Yes**, once fraction terms exist on the board (Fraction equations card; not built).
No opposite box	number of x-terms with a negative coefficient	—	**Yes**, but 0 or 1 on Flip it (binary).
x on the left	0 or 1	never	**Yes**; binary.
Each side simplified	parentheses left + like-term pairs left, both sides	some cleared	**Yes in principle**; needs a board with parentheses (Distribute card; not built). This is Karl's own example and the best fit for yellow.
Solved for x	0 or 1	never	**Yes**; binary.

Blind spots: - **On the cards that exist (one-step), almost every light is binary.** Yellow would appear only on Divide L5 (One box) and nowhere on Add. The idea pays off on Two-step and Distribute cards, which are not built. An experiment on today's cards can't test yellow. - **"Red = nothing done yet" changes what red means.** Today red means "that move made it worse" (loop §5; decision log "Three states: green, neutral, red"; Add spec §3E), and "a legal move is never red" (loop §8). Under idea 2 every light starts red on a fresh problem, so the screen opens with a column of red, and "you made it worse" needs a new signal (a flash? the line alone?). Conflict to settle (Q5). - **A light whose count is already 0 at the start** (No fractions on `x + 8 = 11`) would be green from the first moment. That fits "lights stay on when done" but means some lights are green before the student does anything. - **The unsimplified side.** On Add the other side is left as `11 + −8` so the answer is hidden. "Each side simplified" counted literally would stay yellow until the student types x. The count has to ignore the side that holds the answer, as Fewer terms already does (Add spec §3E).

Part 2, report 3: preview on hover / first tap, and blanks filled in place of Go

Preview (idea 4): large change. - **What exists to reuse:** the equation row builder (`equationRow`, `termsOf` in `solveScreen.js` and `lightScreen.js`), the work-row format that already writes the amount in blue on both sides (`x + 8 + −8 = 11 + −8`, `add-11`; ⅓(3x) = ⅓(12), `div-10`), and the move icons' visual language (dashed blue box, blue `n`). - **What is new:** 1. A **ghost row** under the newest equation: the work row with dashed empty boxes in place of the amount, drawn greyed. One per move type: `+ ▢` on both sides (Add), `▢( … )` (Multiply, not built), `1/▢ ( … )` (fraction), the sides swapped (Trade places). Inferred effort: a renderer per move, sharing the row builder. 2. **Reserved room for it.** Loop §2 forbids reflow, so the stack must hold an empty preview slot. On Add that makes a page that already overflows by 187 px worse (CURRENT-STATE §1.2), so idea 4 on Add needs the space ideas 5-6 free up. 3. **Hover wiring:** today hover is pure CSS (a tooltip). A preview needs pointer-enter / leave events and keyboard focus (so keyboard users see it too), sent to the view only, not the engine (the tucked pilot's `s.ui` pattern in `tuckedLayout.js` is the model). 4. **Touch: first tap previews, second applies.** New. It **replaces loop §5's touch rule** ("a tap shows its text for a moment before it acts"); the spec would change. Cancelling a preview on touch (tap elsewhere) needs a rule too. - **Conflicts:** - **Divide it's move is not "type a number".** It is five fraction buttons (`div-06`), and the spec insists the menu always shows all five (Divide §1, §3B). A `1/▢` blank to type into replaces those buttons. Karl's message describes "1/blank on both sides", so this is a real change to Divide's spec (Q7). - **Flip it's move has no amount** (one tap, no Go: Flip §3B). Its preview is the whole flipped row; "first tap previews" then makes Flip a two-tap move. - **Locked moves.** Today the icons are locked until **Use a move** (`add-03`). If the preview is meant for exploring ("what would this do?") before deciding, it needs idea 6 (no choice gate). If the choice stays, the preview only appears after Use a move.

Blanks filled in the equation; Go merged into Next (idea 5): large change. - **What exists:** Add types the amount into a separate row "Add [ ] to both sides" and presses Go (`add-09`, `add-10`); the engine action is `go` (`addBothMoves.js:393`). On the loop, `commit()` writes the rows and `foldNext` already means "Next commits the move" (`solveLoop.js`, `div-36/div-37`), with Undo reaching only the unfolded move: very close to "result under a line, Undo, Next commits". - **What is new:** 1. The blank lives **inside the preview row** and becomes the input (keyboard and pad focus routed to it; today the amount and `x =` share the pad through a `typing` flag). 2. **"Filling the last blank shows the result"** needs a trigger. With a multi-digit amount (`–12`), when is the blank "filled"? Options: Enter, a short pause, or a small ✓ next to the blank. A timer breaks loop §2 ("nothing advances on a timer"). Inferred: this needs a decision (Q3). 3. **Two blanks or one?** "Plus blank on both ends, you fill in those numbers" reads as two blanks. If the student types both, a mismatch (`+8` left, `+5` right) is possible on the bare screen; today that lesson lives only in L1 and the scaffold. If one blank mirrors to the other side, it is one action as loop §1C says (Q1). 4. **Add has no fold.** On Add the result appears at once and there is no Next after a move; this has to be added or Add moved onto the loop (headline 1 in CURRENT-STATE). - **Conflict: "at later levels the system combines automatically."** On the move that leaves x alone, combining the other side **is the answer** (`x = 11 + −8` → `x = 3`). Loop §1C and §2 say the other side stays unsimplified so the answer is never shown before it is typed. And "at early levels the student types the combined result" makes the student type the answer into the equation row, which duplicates the `x =` row. Needs a rule (Q2): e.g. the student combines only x's side; the other side is never combined.

Part 2, report 4: no Guess x / Use a move choice; `x = [ ]` always available

Code (medium): the choice is the `path` field (`null` / `'guess'` / `'move'`). On the loop it gates `answerLive`, `blockMove` and the line's buttons (`solveLoop.js`; `solveScreen.js:19` draws the buttons). On Add the same field sits in `addBothMoves.js` (`case 'path'`). Removing the choice means: no buttons, moves always live, `x =` always live, and the substitution logic keyed on "what kind of answer is this" (`kind: guess | value`) decided by whether x is alone, which it already is (`solveLoop.js`, `case 'check'`).

Conflicts: loop §1A (the choice is the loop's first rule), §1B's guess flow ("a wrong guess … Undo … the needed move highlighted"), conformance item 1 ("One prompt"), Add spec §1 "Choosing a path", Divide §3B, Flip §1. Every card spec and the checklist would change. The loop spec's Open 5 (counting guess-vs-move choices as teacher data) disappears; the same fact can be recorded as "typed a value before any move".

The forced-move rule (loop §1B) without a choice. Karl's proposal: while a move is forced, the field greys with a short message and the needed move pulses. In code this is small: `answerLive()` returns false while `mustMove` holds and no move has been made, and the line or a

note under the field says why. **It works**, with two blind spots: - **"The needed move pulses" points at the answer.** On Divide L5 (`6x`), pulsing *Take a fraction* is fine, but pre-selecting a fraction would not be. Pulse the move, not the amount. - **Today's forced problem is the problem after a correct first guess.** With the field always there, mental-math students will guess first on every problem, so every other problem is forced. That is what the rule intends, but worth watching in the first test.

Keyboard and pad routing becomes the real design question: if the `x =` field is always live and a move's blank is open, where do typed digits go? Today it follows `typing` (a tap on a field). Needs: the open blank wins until the move is applied or cancelled.

Part 2, report 5: what would be touched, and the risk to saved progress

The loop spec (`SPEC-SOLVE-IT-LOOP.md` rev 1.2): - §1A the choice (idea 6) · §1B wrong-guess flow and forced-move rule (idea 6) · §1C "select, type, Go" and "Why? boxes judged only after Go" (ideas 2, 5) · §2 the screen list (Go in the line, help row "at the bottom of the equation mat", `x = [_]` row in the stack, "only one thing highlighted": a preview plus a pulsing move is two) (ideas 4-7) · §3D operation blue (idea 8) · §5 move panel: greyed-move tap message, touch rule, three light states, "lights judged only after Go" (ideas 2, 3, 4) · §6 scaffold (idea 7) · conformance items 1, 6, 8, 11. - Card specs: Add §1 (bare screen, choosing a path), §3B (the move), §3E (lights), §3G (Go), §3H (icons); Divide §1, §3B (fractions row, five buttons), §4 (lights); Flip §3B (one tap, no Go), §3E.

Save codes: no risk from these ideas. A save code is one bit per finished level (`src/engine/progress.js:147-269`, code v14). The saved level-in-play is only `{pack, level, seed, index}` (`src/main.js:111`), so a problem in progress restarts fresh; no move, light or layout state is saved. **Risk appears only if a level is added, removed or changes meaning** (for example, if Divide L1 changed or a level's content shifted, an old "done" bit would credit different content). A flag-gated experiment that keeps the level counts has **no effect on saved progress**.

The class sheet: each finished problem is logged with tags, wrongs, stuck and Teach me counts (`src/lightStore.js:24`). Ideas 4-6 change nothing recorded; dropping the choice removes the implicit "guessed first" behaviour unless it is recorded another way.

Count it / Build it: no shared code with the Solve it screen, apart from the global stylesheet and the small `html()` helper. `lightScreen.js` is imported only by Solve it files (`addBothPlay`, `numericScreen`, `divideScreen`, `divideNumericScreen`, `solveScreen`, `tuckedLayout`). **But** `src/style.css` is one 3,259-line file where `.lb-stack`'s height alone is set six times (`:2868, 2888, 2892, 2980, 2991, 3045`). Layout changes should be scoped to a flag class to avoid surprises. And **Count it's "Flip It"** (`src/packs/index.js:4`) shares its name with the planned Solve it card.

Part 2, report 6: the smallest first experiment

Updated after Karl's answers (Q1-Q4): Add to both sides, Levels 2-3, behind a flag (`?ui=lights`), ideas 4 + 5, with the choice dropped (idea 6). My first recommendation kept the choice; Karl chose to drop it in the experiment too, so idea 6 is part of the build. *Inferred, with reasons:* - **Why Add, not Divide:** Karl's own first example is Add's `+ ▢` on both sides; Add's move already takes a typed amount, so "fill the blank" is a change of *where* the student types, not *what*. Divide's move is five buttons, so idea 5 there means first redesigning the fraction choice (Q7). **The cost:** Add is not on the shared loop, so the experiment lives in Add's own reducer and screen. That's acceptable for a throw-away experiment; not for the final build. - **Why L2-L3:** one term next to x, one move per problem, no Trade places (L5) and no scaffold-only features in play. Every problem is generated and verified, so "one equation" becomes "one level's five problems", which gives a teacher five tries per student instead of one.

What it would do (description only; reusing existing pieces): 1. With no choice in the line, the Add icon is live from the start; hovering it (or focusing it, or the first tap on touch) draws a greyed preview row under the equation: `x + 8 + ▢ = 11 + ▢`, the boxes dashed blue (row builder from `lightScreen.js`; blue from the icon). 2. Clicking (second tap) solidifies the row; the first box takes the typing; **the second box mirrors it** (one action, loop §1C) (decided, Q1). 3. **Enter or a small ✓ by the blank applies it** (no timer; decided, Q3). Only x's side cancels on its own; the other side is never combined by the system (decided, Q2). The engine's existing `go` runs (`addBothMoves.js:393`, `lightBoard :231`), so the result rows and lights are exactly today's. 4. The result appears under a rule line with **Undo**; a green **Next** commits. Add has no fold, so Next is new on this card; it can simply clear Undo and keep the rows (no fold) for the experiment. 5. **The `x = [ ]` field is always live** (no Guess x / Use a move). Typed digits go to an open blank first, then to `x =`. A correct guess at the first state still makes the next problem a forced move: there the field greys with a short message and Add pulses (Karl's proposal). Everything else (the check, I'm stuck, Teach me) stays as built, with the **flag on the screen root**, so nothing changes without `?ui=lights`. 6. **Layout:** to fit 1366×768 it must also drop the "Add [ ] to both sides" row and move Undo into the result area, which frees about 136 px of Add's 187-px overflow (CURRENT-STATE §1.2). The help row stays where it is (below the fold) for the first test.

Which of ideas 1, 2, 3, 6 and 7 the experiment needs: - **1 and 3: already there,** no work. - **6: needed (decided, Q4).** The experiment drops the choice: moves live and `x = [ ]` open from the start, with the forced-move rule as above. The layout part of idea 6 (I'm stuck and Teach me beside `x =`, Undo and Start over as corner buttons) is optional for the test but it also frees the space Add needs to fit 1366×768. - **2: can wait.** On Add the count is 0 or 1, so yellow would never show; it can't be tested here. - **7: can wait.** Add L2-L3 reserve no scaffold space on the bare screen. - **8: can wait,** but if Karl wants the preview's blank in Add's own poster colour, decide before the build (Q8).

What to watch with real students: do they hover before choosing (or ever, on a touch Chromebook)? Does the preview help choose the amount, or do they still type the number next to x? How many type the same sign instead of the opposite (red Fewer terms) compared with today? Do they find Next after the result, or wait?

Idea 8: the operation colour

Where it is set (observed): the icons' blue is a constant in `lightScreen.js:32` (`BLUE = '#0072b2'`, with `LIGHT = '#6fb6e0'`); the amount added is `.lb-num.is-added` (`style.css:2846`); the fraction in work rows and the check is `.is-blue` (`style.css:2970`); the line colours fraction glyphs blue (`solveScreen.js:16`). **The same `#0072b2` is also the "click here" colour:** pulsing rings, the drop target, hint outlines, the plug-back-in row (`style.css:2786, 2794, 2834, 2881, 2966`).

What a colour per move would take: one colour token per move (CSS variables, for example `--move-add`), used by that move's icon, its amount in the work rows, the preview's blanks and its fold label; and separating "click here" from "operation" so a pulsing ring is not mistaken for a move. Small to medium in code. Each icon is already a distinct shape, so "colour is never the only signal" is satisfied for the icons; for the amount in a work row, the shape is the notation itself (+▢, `n(…)`, `1/n`).

Conflicts: loop §3D ("the operation is blue wherever it appears") and §3C ("the accent must be unique: not green or red, not the blue of the icons"). **The colour budget is tight:** green and red are taken by the lights and Balanced; purple `#7b3fb0` by the step highlight; magenta `#b8508f` by opposites; green `#009e73` by Divide's positive counters; blue by "click here". Eight moves would need eight more distinguishable hues that also print in black and white. Inferred: it is workable for the two or three moves on any one card, hard for all eight at once (Q8).

Part 3: questions for Karl

Asked in the session; answers are recorded below as they come in.

Batch 1 (answered 2026-10-13)

#	Question	Karl's answer	Recommended?
Q1	Idea 5 on Add: one blank mirrored to the other side, or two typed separately?	**One, mirrored.** The student types in the left box; the right fills with the same number.	yes
Q2	Idea 5: what does the system combine?	**x's side only.** The other side is never combined automatically; typing x is the student's combining. The answer is never shown before it is typed.	yes

#	Question	Karl's answer	Recommended?
Q3	Idea 5: what makes the result appear?	**Enter or a small ✓ beside the blank.** No timer.	yes
Q4	First experiment: keep or drop the Guess x / Use a move choice?	**Drop it in the experiment too.** Moves always live, `x = [ ]` always open, the forced-move rule as Karl proposed.	no (I recommended keeping it)

Batch 2 (answered 2026-10-13)

#	Question	Karl's answer	Recommended?
Q5	Idea 2: what does red mean?	**Not done = grey, red = worse.** Not started = outline/grey; partly = yellow ◖; done = green ✓; red stays "that move made it worse".	yes
Q6	Idea 1: icon always beside its light, or revealed by clicking a red light?	**Always beside** (as built).	yes
Q7	Divide it: replace the five fraction buttons with a `1/▢` blank?	**Yes, a `1/▢` blank.** The Divide spec's "menu always shows all five fractions" (§1, §3B) has to change.	no (I recommended deciding after the Add test)
Q8	Idea 8: a colour per move?	**Later, with a palette pass.** Keep blue for the first test; first split "click here" from "operation", then pick move colours that print in black and white.	yes

What the answers change

- **Idea 2 is now:** grey (not started), yellow ◖ (partly), green ✓ (done), red × (worse). This removes the conflict with loop §5 and the decision log over red; the spec still needs the fourth state and the counts.
- **Idea 5 no longer conflicts** with "the answer is never shown" (Q2).
- **Idea 6 is in the first experiment,** so the experiment changes loop §1A/§1B behaviour behind the flag; the loop spec should get a note that §1A is under test.
- **Divide it** will need a spec revision for the `1/▢` blank (Q7) before any Divide build. Open points for it: what a non-splitting denominator does (today: a red failed row), and whether `1/1` or numbers above 6 are refused.
- **Still open:** a definition of "Smaller numbers" as a count (report 2).